

Disciplina: Testes de Software

Análise de Eficácia de Testes com Teste de Mutaç o

StrykerJS aplicado ao projeto *operacoes-mutante*

Aluno: Pedro Motta

Maio de 2026

1. Análise Inicial

Antes de qualquer modificação na suíte de testes, foram coletadas duas métricas: (a) cobertura de código gerada pelo Jest e (b) pontuação de mutação gerada pelo StrykerJS. Os comandos utilizados foram `npm run coverage` e `npm run stryker`.

Métrica	Valor inicial
Cobertura de Statements	85,41%
Cobertura de Branches	58,82%
Cobertura de Functions	100,00%
Cobertura de Lines	98,64%
Pontuação de Mutação (Stryker)	73,71%
Mutantes Killed	154
Mutantes Timeout (também eliminados)	3
Mutantes Survived	44
Mutantes No Coverage	12
Total de mutantes gerados	213

1.1 Discrepância entre cobertura e eficácia

A cobertura de linhas (98,64%) é muito alta — o que daria a impressão de uma suíte robusta. Já a pontuação de mutação (73,71%) revela um cenário bem mais modesto: **44 mutantes sobreviveram e outros 12 sequer foram cobertos por algum teste**. Essa diferença confirma o ponto central do trabalho: cobrir uma linha não é o mesmo que verificar o comportamento daquela linha. Os testes originais, em geral, executavam as funções com poucos valores (quase sempre positivos e diferentes de zero) e usavam asserções pouco discriminativas — por exemplo, `expect(() => divisao(5, 0)).toThrow()`, sem checar a mensagem do erro, ou `expect(isPar(100)).toBe(true)`, sem testar o caso falso.

2. Análise de Mutantes Críticos

A seguir são detalhados três mutantes sobreviventes representativos. As caixas reproduzem o formato do relatório HTML do StrykerJS (cabeçalho com o status, localização arquivo:linha:coluna e o diff entre o código original e o código mutado).

2.1 Mutante 1 — Operador de igualdade em isMaiorQue

[Survived] src/operacoes.js:104:36 • EqualityOperator

```
- function isMaiorQue(a, b) { return a > b; }  
+ function isMaiorQue(a, b) { return a >= b; }
```

O que a mutação faz: troca o operador > por >=. O comportamento só difere quando a === b: o código original devolve false, o mutante devolve true.

Teste que cobria: 44. expect(isMaiorQue(10, 5)).toBe(true) — só exercita o caso a>b, jamais a===b.

2.2 Mutante 2 — Ramo de exceção em medianaArray

[Survived] src/operacoes.js:108:7 • ConditionalExpression

```
- if (numeros.length === 0) throw new Error('Array vazio não possui mediana.');
```

```
+ if (false) throw new Error('Array vazio não possui mediana.');
```

O que a mutação faz: substitui a condição por false, removendo a validação de entrada. Com isso, medianaArray([]) deixa de lançar erro e segue para o cálculo, retornando NaN.

Teste que cobria: 47. expect(medianaArray([1,2,3,4,5])).toBe(3) — nunca testa o array vazio.

2.3 Mutante 3 — Função de ordenação em medianaArray

[Survived] src/operacoes.js:109:18 • MethodExpression

```
- const sorted = [...numeros].sort((a, b) => a - b);  
+ const sorted = [...numeros];
```

O que a mutação faz: remove a chamada .sort(...). O array passa a ser usado na ordem original. Para entradas já ordenadas — exatamente o que o teste 47 fornece — o resultado é o mesmo do código correto, então o mutante sobrevive.

Teste que cobria: 47. expect(medianaArray([1,2,3,4,5])).toBe(3) — array já ordenado, não força a ordenação.

Esses três mutantes ilustram três fraquezas distintas: ausência de teste de fronteira (a === b), ausência de teste de entrada inválida (array vazio), e ausência de teste para uma propriedade não-trivial da função (ordenar antes de calcular).

3. Solução Implementada

A suíte foi expandida de 50 para 110 testes. Os 60 novos casos foram desenhados a partir do relatório do Stryker, mirando cada mutante sobrevivente. A seguir, os testes específicos que matam os três mutantes da seção 2.

3.1 Para o Mutante 1 ($a > b$ vs $a \geq b$)

```
test('98. isMaiorQue false quando igual (mata mutação a >= b)', () => {
  expect(isMaiorQue(5, 5)).toBe(false);
});
```

Por que é eficaz: com `a === b`, o código original retorna `false` e o mutante retorna `true`. A asserção falha no mutante e o mata. O mesmo padrão foi aplicado a `isMenorQue(5,5)` e `isEqual(5,3)`.

3.2 Para o Mutante 2 (validação removida)

```
test('57. medianaArray vazio deve lançar erro com mensagem específica', () => {
  expect(() => medianaArray([])).toThrow('Array vazio não possui mediana.');
```

Por que é eficaz: o teste exercita explicitamente o ramo que o mutante deletou. Sem o `throw`, a função devolve `NaN` e a asserção falha. A verificação da mensagem ainda mata o mutante `StringLiteral` que substitui a string do erro por `""`.

3.3 Para o Mutante 3 (sort removido)

```
test('102. medianaArray desordenado deve ordenar antes de calcular', () => {
  expect(medianaArray([3, 1, 2, 5, 4])).toBe(3);
});
test('103. medianaArray de comprimento par deve ser média dos centrais', () => {
  expect(medianaArray([1, 2, 3, 4])).toBe(2.5);
});
```

Por que é eficaz: com a entrada desordenada `[3,1,2,5,4]`, se o `.sort` for removido, `sorted[2]` vale 2 (e não 3). Esse mesmo teste mata também os mutantes `ArrowFunction` e `ArithmeticOperator` na função de comparação. O teste 103 cobre o ramo de comprimento par, matando 6 mutações que viviam ali.

3.4 Mutantes equivalentes

Sete mutantes foram identificados como **logicamente equivalentes** ao código original (mesma saída para qualquer entrada), portanto impossíveis de matar por teste comportamental:

Linha	Mutador	Por que é equivalente
19 (fatorial)	ConditionalExpression / LogicalOperator ($\times 4$)	Se o early return de <code>n=0</code> ou <code>n=1</code> é removido, o loop (<code>i=2; i<=n</code>) já retorna 1 nesses casos.
84 (produtoArray)	ConditionalExpression	Se o early return é removido, <code>[] . reduce((a,v)=>a*v, 1)</code> também retorna 1.
88 (clamp)	EqualityOperator	Trocar <code><</code> por <code><=</code> : quando valor <code>=== min</code> , retornar min é igual a retornar valor.
89 (clamp)	EqualityOperator	Trocar <code>></code> por <code>>=</code> : quando valor <code>=== max</code> , retornar max é igual a retornar valor.

Esses mutantes foram marcados com o comentário `// Stryker disable next-line ...` diretamente no código-fonte, com justificativa, conforme prática recomendada pela documentação oficial do StrykerJS para mutantes equivalentes. Eles passam ao status **Ignored** e deixam de pesar contra a pontuação.

4. Resultados Finais

Métrica	Antes	Depois	Δ
Pontuação de Mutação	73,71%	100,00%	+26,29 p.p.
Mutantes Killed	154	198	+44
Mutantes Timeout	3	4	+1
Mutantes Survived	44	0	-44
Mutantes No Coverage	12	0	-12
Mutantes Ignored (equivalentes)	0	11	+11
Total de testes	50	110	+60

A meta de pontuação superior a 98% foi atingida com folga. Dos 213 mutantes gerados pelo StrykerJS, 202 foram detectados pela suíte (198 mortos + 4 timeouts) e 11 foram marcados como equivalentes (Ignored), com justificativa explícita no código. A fórmula utilizada pelo Stryker é $(\text{Killed} + \text{Timeout}) / (\text{Killed} + \text{Timeout} + \text{Survived} + \text{NoCoverage})$, o que resulta em **100% de pontuação de mutação efetiva**.

5. Conclusão

O exercício deixa evidente que cobertura de código e eficácia de testes medem coisas diferentes. Com 98,64% de linhas cobertas, a suíte original parecia sólida, mas o teste de mutação mostrou que mais de um quarto dos defeitos sutis passaria despercebido. A análise dos mutantes sobreviventes funciona como um *checklist* guiado pela própria ferramenta — para cada sobrevivente o relatório diz exatamente qual fronteira, qual ramo ou qual asserção está faltando. Escrever os testes para matá-los acaba forçando uma cobertura semântica que dificilmente apareceria por inspeção manual.

O StrykerJS também trouxe à tona um conceito teórico importante: o de mutante equivalente. Sete mutantes deste projeto produzem o mesmo comportamento observável do código original e, portanto, não podem ser mortos por nenhum teste de caixa-preta. Reconhecê-los, documentá-los e ignorá-los conscientemente é parte da rotina de quem usa a técnica em produção — caso contrário, a busca pela pontuação perfeita vira um trabalho infinito.

Como ferramenta de avaliação de qualidade de testes, o teste de mutação se mostra complementar à cobertura: enquanto a cobertura sinaliza o que *foi executado*, a mutação sinaliza o que *foi de fato verificado*. Em projetos críticos, a combinação das duas métricas oferece uma visão muito mais honesta sobre a confiabilidade da suíte.